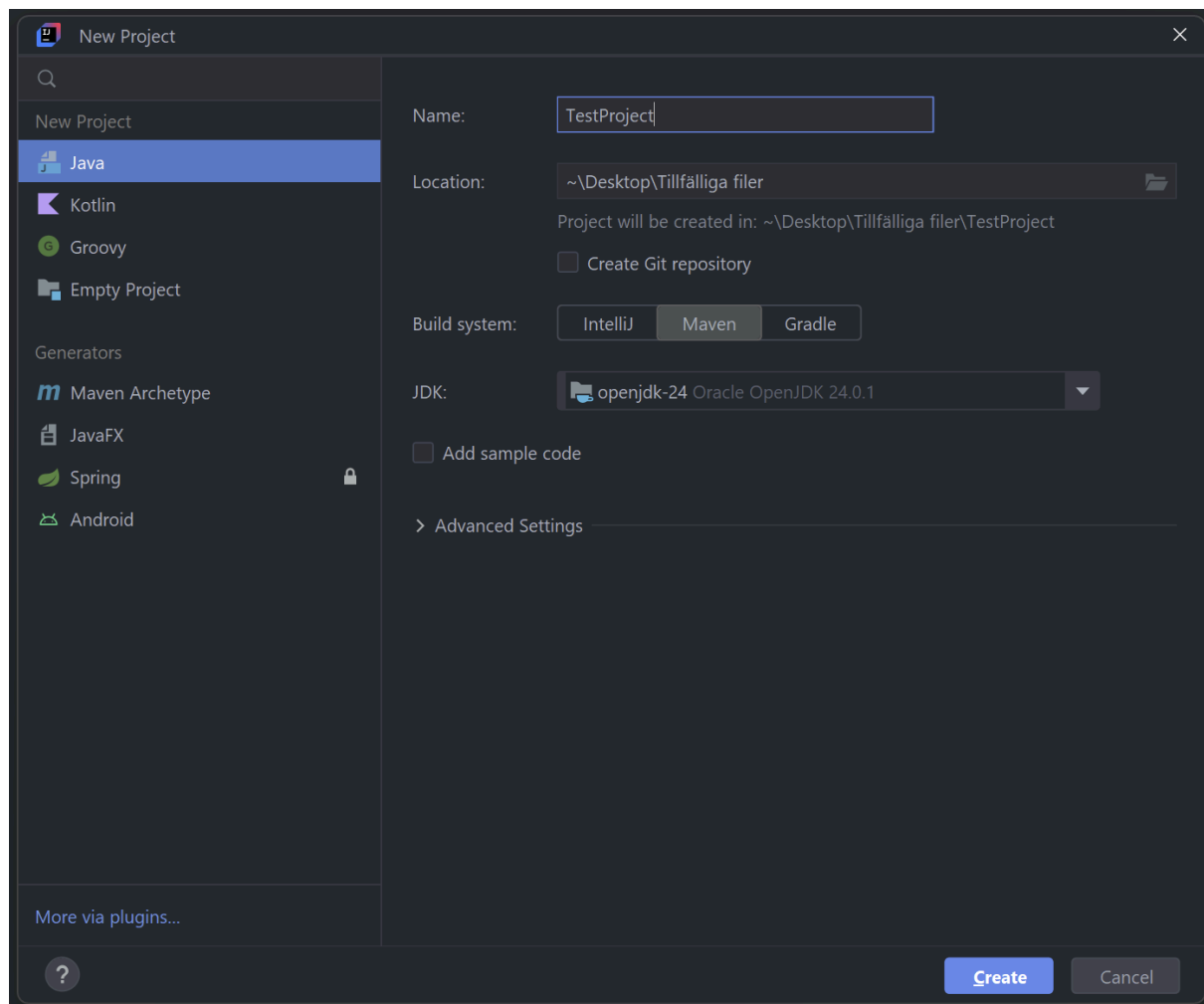


Test-Driven Development med Maven och JUnit 5

1. Använd Maven som byggsystem för ditt projekt

Klicka på New Project... och välj Java som språk. Välj sedan Maven som byggsystem:

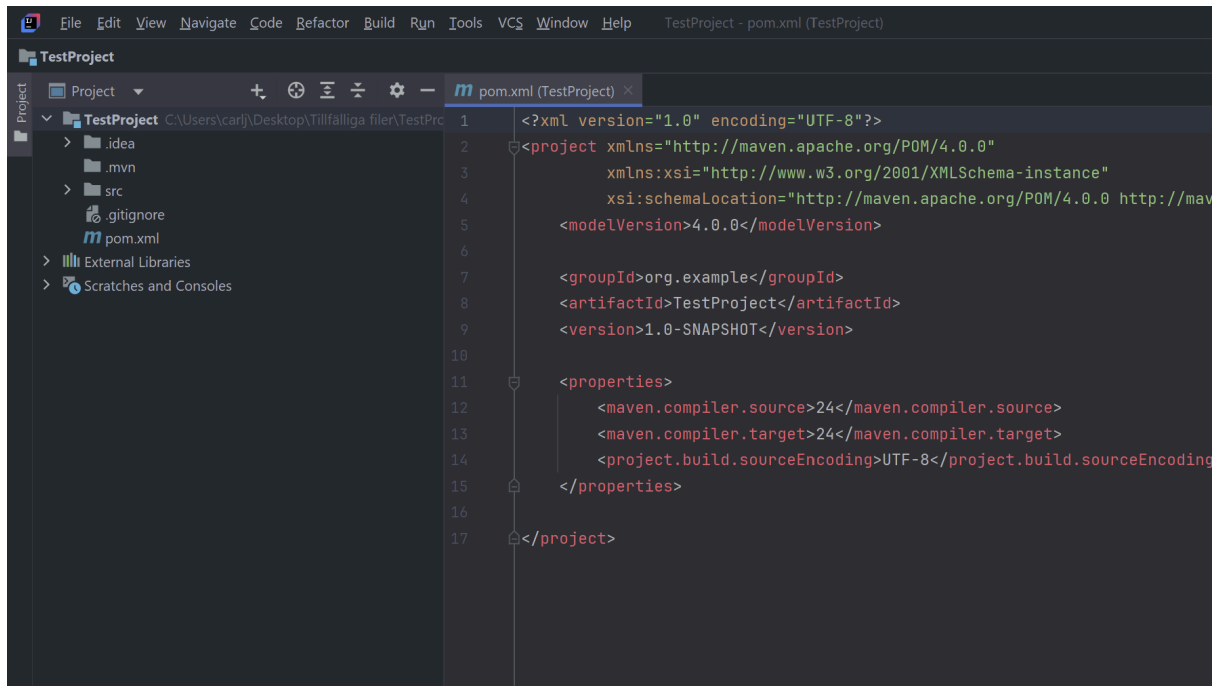


Maven är ett byggverktyg som gör att vi kan hålla koll på så kallade beroenden i lite större projekt. Ett beroende är extern kod som vi behöver importera för att använda i vårt program: det vill säga, det är kod som inte är en del av vår egen källkod som vi skriver.

Klicka ok så att ett nytt projekt skapas.

2. Vad pom, xml och beroenden är för något

IntelliJ kommer automatiskt att öppna någonting som heter pom.xml:



Pom-filer (Project Object Model) är konfigurationsfiler för Maven skrivna i XML. De berättar bland annat vad projektet heter, vilken version av Java det använder och vilka bibliotek det är beroende av. XML är en form av märkningsspråk som påminner om HTML. Det är vanligt att använda i sammanhang där man behöver strukturera data och lagra metadata. I det här fallet har Maven redan fyllt i några XML-taggar åt oss. Låt oss kika på dessa lite snabbt:

groupId-taggen berättar vem som äger projektet. Att det börjar med just org. (kort för organisation) är bara en konvention: vi kan döpa det till vad vi vill - så länge namnet vi väljer använder punktseparatorn någonstans. Om man publicerar projekt offentligt måste de ha unika id:n, men vi behöver inte bry oss om detta just nu.

Låt oss ändra vårt groupId till my.testProject i stället, bara för att vi kan (detta känns som en utmärkt anledning att göra det mesta i livet!) :



Ytterligare två taggar är bra att hålla koll på: **artifactId** berättar vad projektet i IntelliJ heter. Denna tagg bör (men måste tekniskt sett inte) matcha projektnamnet. **version**-taggen berättar vilken projektversion vi har. Lämna båda dessa taggar orörda.

3. Lägga till JUnit 5 bland dina beroenden

Det är dags att lägga till ett JUnit-beroende i den här pom-filen. Först behöver vi skapa en sektion i filen som heter dependencies. Vi gör detta genom att skriva in taggarna manuellt:

```
11  <properties>
12    <maven.compiler.source>24</maven.compiler.source>
13    <maven.compiler.target>24</maven.compiler.target>
14    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15  </properties>
16
17  <dependencies>
18    |
19  </dependencies>
20 </project>
```

Därefter behöver vi skriva in en dependency. Exakt vilket beroende man vill lägga till är ju dock inte alltid så lätt att veta på förhand! Ofta hittar man dessa listade på nätet, t.ex. genom att söka i Mavens databas.

(Att fråga AI rekommenderas ej eftersom den sällan har koll på rätt versioner, skillnader mellan olika paket, osv. Detta uppdateras och förändras på flera olika ställen: dels dyker det upp nya versioner frekvent, men man behöver också fundera över vilken version man kan räkna med att IntelliJ:s Maven-plugin är kompatibel med, och dylikt).

Det bästa vore om programmet hanterade sånt här åt oss automatiskt, men tyvärr finns det inget bra sätt att söka efter och sedan lägga till beroenden i varken Maven eller Gradle, som är de två mest populära byggsystemen för Java. IntelliJ skulle må bra av en packet manager av samma klass som NuGet som finns i t.ex. Visual Studio, men vi får tyvärr klara oss själva tills någon väljer att skapa en sådan.

I det här fallet vet jag dock på förhand vilken kod vi behöver skriva in, och ni kan helt enkelt kopiera mig:

```
17  <dependencies>
18    <dependency>
19      <groupId>org.junit.jupiter</groupId>
20      <artifactId>junit-jupiter</artifactId>
21      <version>5.12.2</version>
22      <scope>test</scope>
23    </dependency>
24  </dependencies>
```

Klipp-och-klistra-vänlig version av koden:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.12.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Skulle det här av någon anledning inte fungera med 5.12.2 är det ok att använda 5.12.1 eller någon lägre version: 5.12.2 är bara det högsta versionsnummer som fungerar korrekt.

JUnit ligger som vi kan se i ett paket som heter org.junit.jupiter. Att det heter just jupiter på slutet finns det ingen särskild anledning till: det är ett namnval som gjordes vid något tillfälle bara för att peka ut den version av JUnit som riktar sig mot användarna (och inte utvecklarna som skapat JUnit-testen).

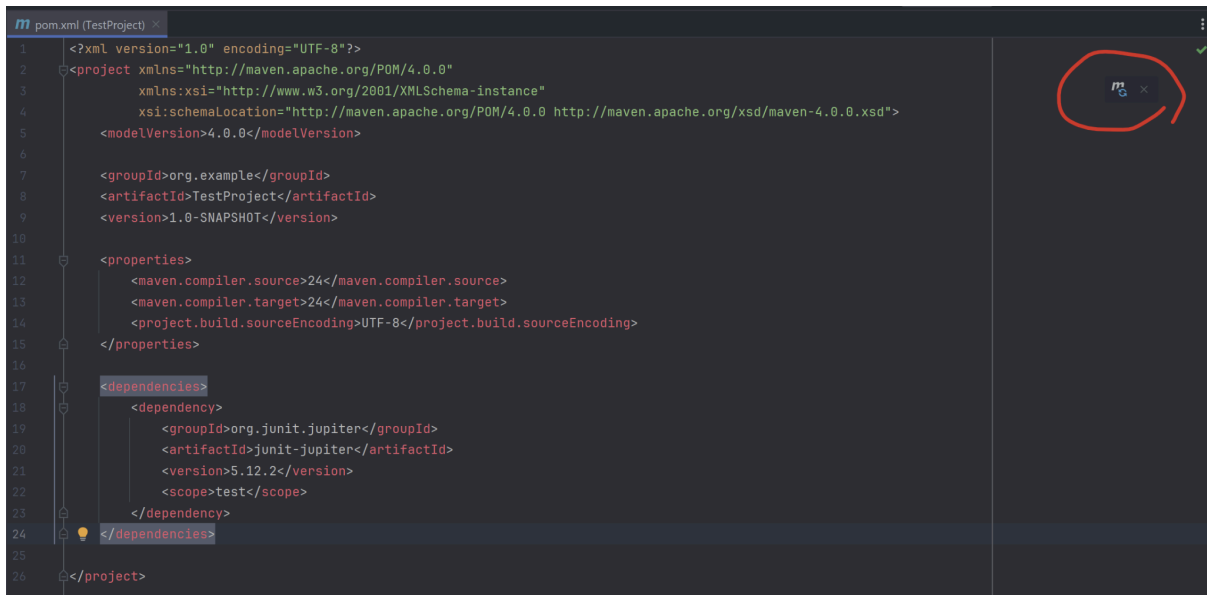
Version 5.12.2 är inte den senaste versionen av JUnit, men det är den högsta som Maven i IntelliJ är kompatibelt med. Om vi försöker lägga till ett beroende för t.ex. 5.14.4 kommer vi få ett kompileringsfel eftersom Maven inte förstår vad det är vi ber det att importera.

Skulle programmet inte hitta artifactID junit-jupiter kan man behöva lägga till två separata dependencies i stället för engine och API (Jupiter är ett aggregatpaket som innehåller dessa två, och att lägga till dem individuellt uppnår därför samma funktion):

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.12.2</version>
    <scope>test</scope>
  </dependency>

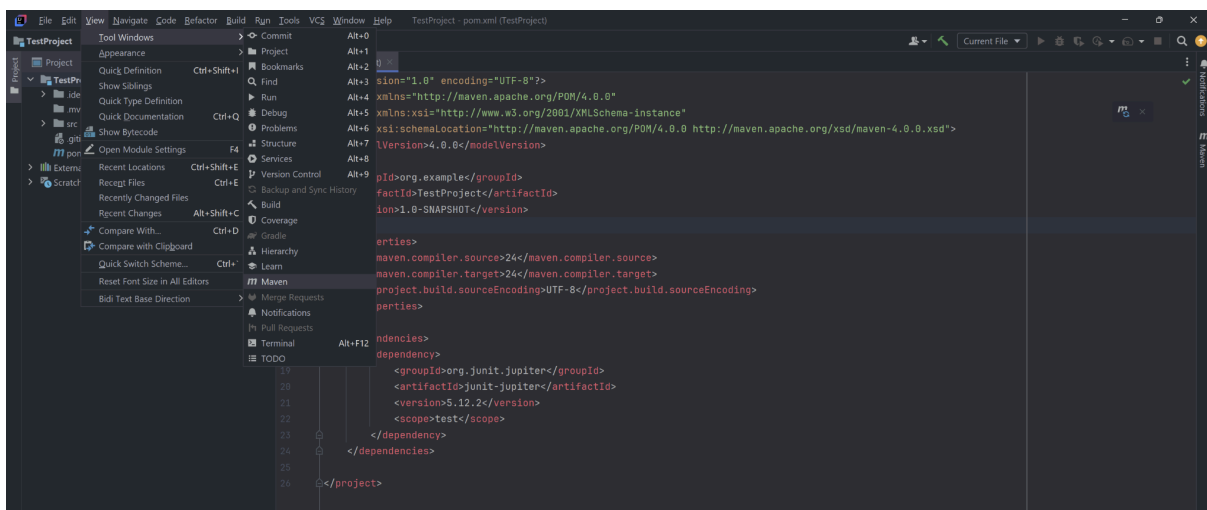
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.12.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

När ni lagt till beroendet bör ni kunna se en liten mavensymbol uppe i högra hörnet som säger "Sync maven Settings" när ni håller muspekaren över den:

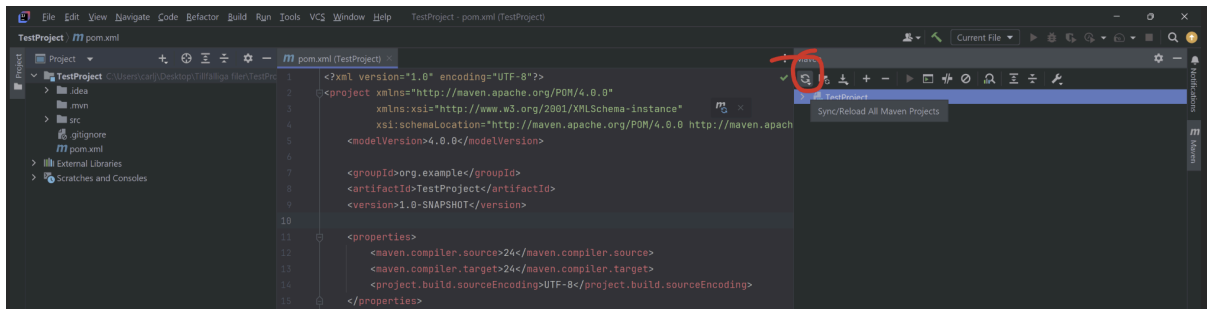


Klicka på den här ikonen så kommer Maven att uppdatera sin konfiguration och hämta de filer vi just bett det om att importera till vårt projekt.

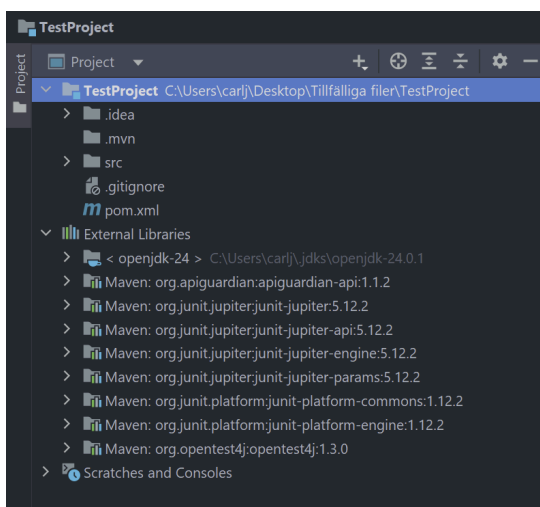
Om ni inte ser synkroniseringssymbolen kan ni också gå in under View > Tool Windows > Maven i menyn:



Ni kommer att få upp en meny till höger där ni ser ert projekt. Ovanför det i verktygsraden finns en liten -symbol: klicka på den och välj "Sync all Maven Projects".



Om allting gått bra så här långt kan du nu öppna projektvyn och förlänga “External Libraries”, och du kommer då se alla filer som Maven nyss importerade (JUnit är lightweight så det här går fort!):

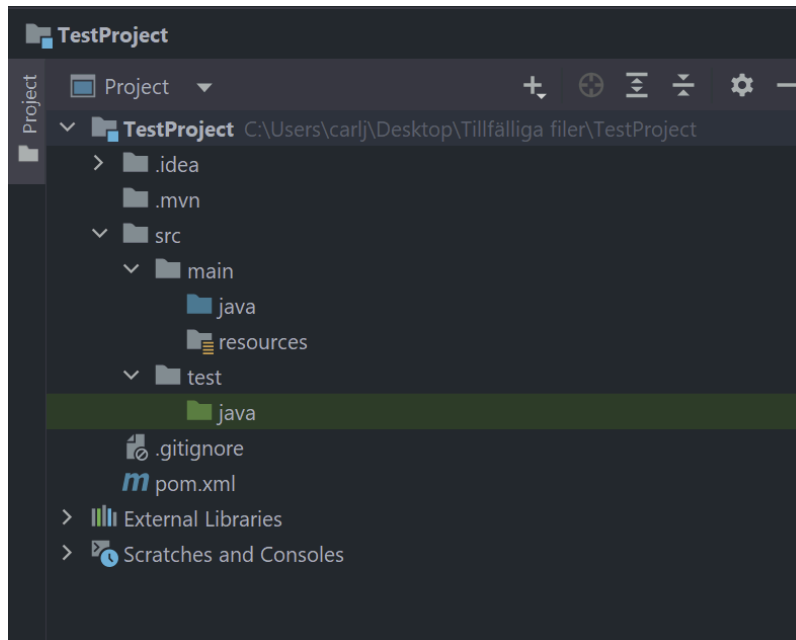


Det andra beroendet som finns i den här listan är ni redan bekanta med: det är ert Java Development Kit som alltid läggs till automatiskt som beroende så fort vi definierar ett JDK för ett projekt.

Det här är vad som gör att vi t.ex. kan skriva `import java.util.ArrayList` och få en ArrayList som vi kan använda oss av - och nu kan vi göra samma sak med JUnit i våra testklasser! Eftersom de här filerna nu ligger som beroenden kan vi importera kod från dem.

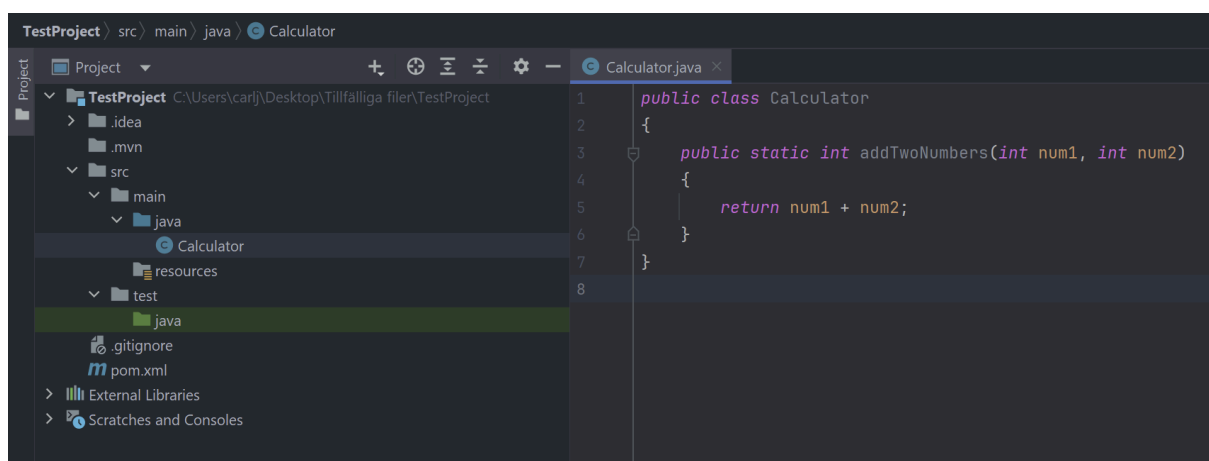
4. Att skapa en testklass

Vi kan nu förlänga src-mappen i projektvyn. Det första vi upptäcker är att Maven automatiskt har skapat lite fler packages än de vi får när vi använder IntelliJ som template för vårt byggsystem. Det finns en main och en test:



resources-mappen kommer vi inte att använda oss av och det går utmärkt att radera den om man vill.

Våra källkodsfiler lägger vi alltid i src/main/java i den här strukturen. Låt oss skapa en oerhört simpel Calculator-klass som bara innehåller en metod som summerar två tal:



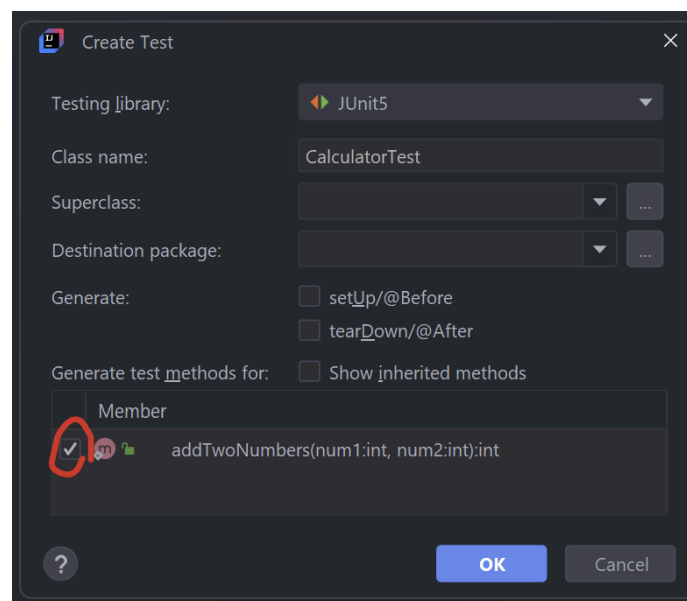
Vi har nu en klass som gör någonting i vårt program! Och för att försäkra oss om att den här klassen fungerar som den ska bör vi nu testa dess metoder, av vilka den bara har en enda.

Vi kan skapa en testklass på två olika vis: vi kan antingen göra det manuellt genom att högerklicka på src/test/java och skapa en ny klass där som vi döper till något i stil med "TestCalculator.java".

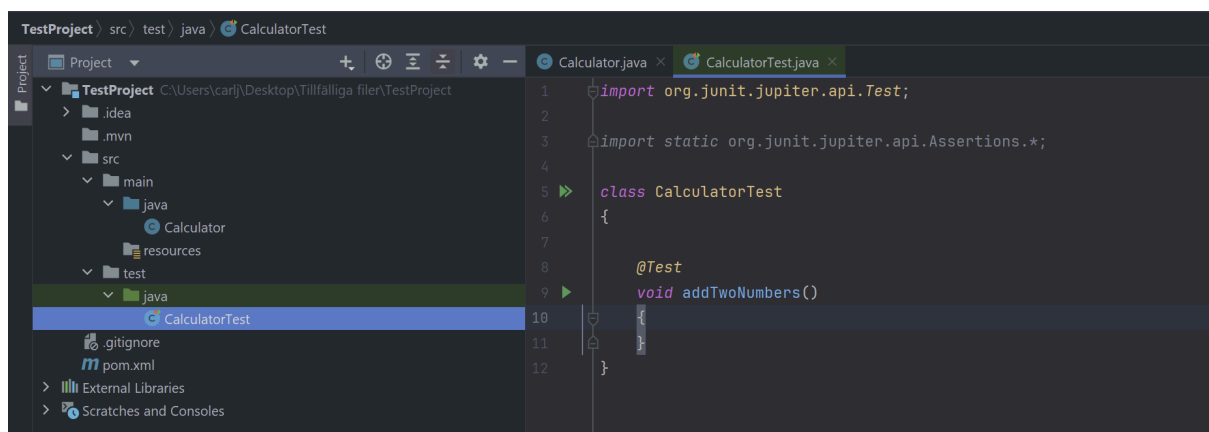
Vi kan också trycka på Ctrl + Shift + T i Windows eller Command + Shift + T på Mac.

Vi kan också helt enkelt gå in under Navigate > Test i menyn. Vi får då upp en prompt som erbjuder sig att skapa själva testklassen åt oss automatiskt, vilket vi tackar för!

Bocka i kryssrutan som säger att vi vill generera en testmetod för addTwoNumbers() och klicka ok:



Programmet har nu autogenererat en testklass åt oss där den redan importerat de saker vi kommer att behöva använda oss av. Dessa importar pekar mot de filer som Maven tankade hem åt oss när vi la in ett JUnit-beroende i pom-filen. Vi noterar även att testklassen automatiskt har lagts till i testpaketet:



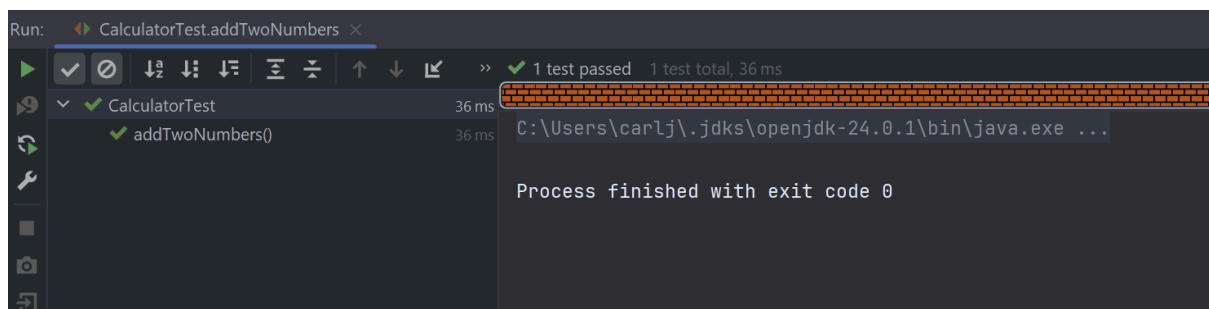
5. Att skriva en testklass

När vi jobbar med test använder vi oss av så kallade **asserts**. En assert är ett påstående vi gör om någonting, och i den här kontexten vill vi göra ett påstående om vad resultatet av att anropa metoden `addTwoNumbers()` med två särskilda nummer ska vara.

Låt oss säga att vi vill testa att lägga samman $5 + 5$: vi vill påstå att det här blir 10 om koden fungerar som den borde, och för detta använder vi någonting som heter `assertEquals()`:

```
8      @Test
9      void addTwoNumbers()
10     {
11         int result = Calculator.addTwoNumbers(5, 5);
12         assertEquals("expected: 10, result");
13     }
14 }
```

Notera att en grön pil dyker upp i vänstermenyn när vi fyller i den här metoden. Om vi klickar på den kör vi testet, varpå vi får upp ett meddelande om att testet lyckades:



The screenshot shows the 'Run' window of an IDE. The top bar indicates '1 test passed' and '1 test total, 36 ms'. Below this, a tree view shows 'CalculatorTest' with a green checkmark and '36 ms', and 'addTwoNumbers()' with a green checkmark and '36 ms'. The main pane shows the command prompt output: 'C:\Users\carlj\jdk\openjdk-24.0.1\bin\java.exe ...' and 'Process finished with exit code 0'.

Vi har nu skrivit vårt första positiva test! Egentligen borde vi ändra namnet från det autogenererade, för det berättar bara vilken metod vi anropar men inte vad vi faktiskt testar. Ett test bör ha ett extremt tydligt namn så att vi vet exakt vad det är som inte fungerar i en lång lista av test. Låt oss ändra namnet till `fivePlusFiveEqualsTen()` i stället för tydlighetens skull:

```
8      @Test
9      void fivePlusFiveEqualsTen()
10     {
11         int result = Calculator.addTwoNumbers(5, 5);
12         assertEquals("expected: 10, result");
13     }
```

Vi har nu en testklass med en testmetod som berättar exakt vad det är vi testar!

När man skriver test räcker det dock inte med att bara använda sig av positiva påståenden: dvs sådana där vi vet vad resultatet bör bli. Vi bör också skriva ett negativt test, där vi gör ett påstående om vad resultatet INTE bör vara. Vi vet t.ex. Att $5 + 5$ inte bör vara 25. För detta använder vi någonting som heter `assertNotEquals()`:

```
8      @Test
9      void fivePlusFiveEqualsTen()
10     {
11         int result = Calculator.addTwoNumbers(5, 5);
12         assertEquals( expected: 10, result);
13     }
14
15     @Test
16     void fivePlusFiveDoesNotEqualTwentyFive()
17     {
18         int result = Calculator.addTwoNumbers(5, 5);
19         assertEquals( unexpected: 25, result);
20     }
```

Vi skriver detta test som en separat testmetod eftersom vi inte vill blanda testerna. Kör **aldrig** två eller flera test i samma metod: dels för att vi vill veta exakt vad som går fel utan att behöva gräva i koden, men också för att test som går fel kan generera undantagsfel och liknande som kanske av misstag fångas i annan testkod och då påverkar den. Tanken med test är att vi vill kunna isolera felen och se direkt vad det är som gått sönder i koden.

Anledningen till att vi skriver både positiva och negativa test för kod är simpel, men viktig att förstå:

Positiva test visar att koden fungerar korrekt när den används korrekt.

Negativa test visar att koden misslyckas korrekt när den används inkorrekt.

Viktig insikt: Även ett negativt test bör ha förutsägbara konsekvenser.

Att skriva test handlar inte bara om att validera ett resultat: det handlar om att försäkra sig om att beteendet för en metod förblir samma när kodbasen utvecklas och förändras. I mer komplexa program är test livsviktiga för att garantera att vi inte har sönder någonting när vi uppdaterar eller redigerar interna delar av ett program.